# DE ID - MDM - Product Change Reporting


- Overview
- Stored Procedure Breakdown
  - 1. Constants
  - 2. Get Other Variables
  - 3. Get Table PK
  - 4. Set Up Column List
  - 5. Build SQL Query
  - 6. Build delete SQL Query
  - 7. Load Changelog
  - 8. Update DELETED records
- Snowflake Notes
- Matillion Notes
- Future Enhancements


This document serves as the internal Data Engineering team-facing information about the Snowflake stored procedure `PLM_DEV.PUBLIC.SP_BUILD_MDM_CHANGELOG`, which populates the table `PLM_DEV.PUBLIC.MDM_CHANGELOG_TBL` and the view `PLM_DEV.PUBLIC.MDM_CHANGELOG_V`. This is not yet in Tableau for end users as it's still in internal validation.

## Overview

Right now there is no way to track changes to the product line that are happening across different systems (Oracle, MPL, Agile, Flex). We needed to build a comprehensive way to capture changes to product data that scales across multiple systems. V1 only includes select MPL tables. V1.1 includes other important MPL tables and basic Oracle data. V1.5 and beyond includes Agile + Flex data (but this is TBD).

Tech Spec - Tech Spec: Product Change Reporting

Requirements Gathering - Requirement Gathering: Product Change Reporting

Tables part of current version 1.1:

| Table | Record Type | Notes |
|---|---|---|
| XXSBC_MPL_SEASONLESS_SKUS | New, Update, Delete | |
| MPL_BIKES | New, Update, Delete | |

| | | |
|---|---|---|
| MPL_BIKE_SKUS | New, Update, Delete | |
| MPL_PRODUCT_LIFECYCLE_DATES | New, Update, Delete | |
| MPL_EQUIPMENT | New, Update, Delete | Table in source has column DELETED as a varchar with possible values: Y, N, NULL, the behavior among these values goes: NULL to N, NULL to Y, Y to N, N to Y, these changes are captured as DELETE whenre NEW_VALUE is Y, other cases are track as UPDATE. |
| MPL_EQUIP_SKUS | New, Update, Delete | Table in source has column DELETED as a varchar with possible values: Y, N, NULL, the behavior among these values goes: NULL to N, NULL to Y, Y to N, N to Y, these changes are captured as DELETE whenre NEW_VALUE is Y, other cases are track as UPDATE. |
| MPL_BIKE_FOBS | New, Update, Delete | |
| ORACLE_ITEMS | New, Update, Delete | |

### Stored Procedure Breakdown

Github Link: [https://github.com/SpecializedBikes/de-snowflake/blob/main/PLM/Stored%20Procedures/SP_BUILD_MDM_CHANGELOG.sql](https://github.com/SpecializedBikes/de-snowflake/blob/main/PLM/Stored%20Procedures/SP_BUILD_MDM_CHANGELOG.sql)

`SP_BUILD_MDM_CHANGELOG` is set up to process one table at a time. It takes the parameters of DATABASE_NAME, SCHEMA_NAME, TABLE_NAME, CHANGELOG_DB and optional parameter DELETED. The script is broken out into 7 different parts: Constants, Get Other Variables, Get Table PK, Set Up Column List, Build SQL Query, Build Delete SQL Query and Load Changelog. I will talk through each one in detail below.

#### 1. Constants

This is where the constants are set, the variables that don't change throughout the procedure. The parameter values are set to their own variables, the table name that we are processing through the changelog is set, as is the changelog table (based on the CHANGELOG_DB param i.e. PLM_DEV for testing or PLM for production). Here we also

define columns we want to exclude from being tracked in the changelog because they are metadata fields or our SCD fields, cols_to_exclude. We also define columns we want to exclude from the calculated fields Change and Change % which track the differences between old and new values for Numbers, Dates, or Timestamps data types, cols_to_exclude_change_diff. These 2 values that are in there currently, DISPLAY_ORDER and UPDATE_USER are set to NUMBER data type but it is not meaningful to track the Change and Change % on them.

#### 2. Get Other Variables

For the table name passed in, we get column information from the DAF configuration table to populate source_system_name, watermark_col, and changelog_flag fields and these are set as variables. Next there is a check to ensure the changelog_flag is set to TRUE for that table. If it's not, an alert gets sent to #alert-snowflake and the procedure exits. Finally, for incremental loading, we get the max change date in the changelog table for the table name passed in. If the table is empty, it defaults to 1900-01-01 to pull in everything.

#### 3. Get Table PK

This part looks up the primary keys that are set for the table being passed in. The primary key must be defined for the table and accessible through the INFORMATION_SCHEMA, or else an alert gets sent to #alert-snowflake and the procedure exits. The primary key is critical in the logic for the LAG function determining the old/new values which is explained in step 4.

#### 4. Set Up Column List

This part uses the INFORMATION_SCHEMA again to get a list of the columns available in the table being passed in AND build some calculated fields. We filter out the columns that are defined in Step 1 in the cols_to_exclude variable, as well as filter out the surrogate key we make for each table that begins with SK_. We also pull in the data type for the column which is used in the calculations for the Change and Change % fields. Column name and data type go into a tuple list that is iterated over later in the procedure.

Next there is a check to see if the table has a field called SKU. If it does, we want to use that value, otherwise the SKU field is only filled in where the table's primary key is SKU. This ensures tables that have SKU field, but it's not the primary key, still get the values filled in.

The next part of this section, and most important to the changelog, is building the wide select statement that uses LAG function to track the old and new values of a column. For each column name in the tuple list, we take the previous value using LAG function and create a

new column for it called PREV_<column_name>, and then we have the newest value as it's own column too.

There is a big calculation for the Change and Change % fields. This uses the data type field to determine how the column should be calculated. The Change and Change % fields only make sense to calculate for Numbers and Dates/Timestamps, so everything else will be NULL. There are some fields that are numeric but do not make sense to calculate how much they changed and by what percent (ex: Display Order or User ID). These fields are in the list of cols_to_exclude_change_diff so we exclude them and any field ending with _ID. The calculations are similar but a little different based on the data type. Number/Floats are pretty straightforward in calculating the differences, same with Dates/Timestamps but we add on a "days" suffix (ex: "90 days" or "-10 days").

Lastly, we calculate the record_type field based on if the PK is already in the data set (UPDATE) or is brand new to it (NEW), and join all of these columns together in a giant select statement.

NOTE: Here is when parameter DELETED is evaluated, if is FALSE execute step 5 if isn't execute step 6

#### 5. Build SQL Query

This is putting everything together into a giant SQL query that will be used when we append the records into the table in the next step. We only want to process NEW/UPDATE record types. If there are no NEW/UPDATE records to load, the procedure exits. The SQL query that it builds initially is extremely wide, with 2 columns for each field (remember how we used LAG to create the previous value and kept the new value). This is not how we want to display the changelog so we then have to unpivot the data. The unpivot is handled separately for NEW and UPDATE records. For NEW records we list the column name as "ALL" because it is a brand new record and all of the columns are technically changed. For UPDATE records, we process each column individually. These are then brought back together with a UNION and appended into the MDM_CHANGELOG_TBL.

#### 6. Build delete SQL Query

We only want to process DELETE record types. If there are no DELETED records to load, the procedure exits. The SQL query evaluates PERSIT table against FULL TEMP table and return the active records that not longer exists in FULL TEMP table.  For DELETED records we list the column name as "ALL" because columns not exists anymore. These are then appended into the MDM_CHANGELOG_TBL.

#### 7. Load Changelog

Inserts the data into MDM_CHANGELOG_TBL using the giant SQL created in step 5. We get the count to print out how many records were inserted.

Note that the first initial load of the changelog contains a bunch of NEW records, obviously, but they are not truly “NEW”. When we push the changelog to PROD and do the first run, we will delete all NEW records so from that day on the NEW records will only be truly new records.

#### 8. Update DELETED records

In persist table we update columns: DELETED = TRUE, SCD_ACTIVE_FLAG=FALSE and SCD_END_DATE= CURRENT_TIMESTAMP () to mark those records as deleted.

for tables MPL_EQUIPMENT and MPL_EQUIP_SKUS updates RECORD_TYPE = DELETE when column = DELETED and NEW_VALUE = Y

and finally, we update column CHANGELOG_LAST_RUN in table ETL_PERSIST_LAYER_CONFIG to register the execution.

### Snowflake Notes

| Database | Schema | Table/View Name | Tableau Datasource Name | Important Distinction | Data Quirks |
|---|---|---|---|---|---|
| PLM_DEV | PUBLIC | MDM_CHANGELOG_TBL | N/A yet | | The stored procedure appends records to the table. |
| PLM_DEV | PUBLIC | MDM_CHANGELOG_V | | Event Tracking information enriches the Changelog within the view. | |

### Matillion Notes

| Matillion Project | Schema | Job Name/Link | Job Notes |
|---|---|---|---|
| PLM | DEV | CALL_MDM_CHANGELOG_QUERY | This job calls the stored procedure to process the table that was passed in. It |

| | | | |
|---|---|---|---|
| | | | is called from MDM_DB_QUERY or MDM_SP_QUERY jobs where the variables get set if changelog_flag = TRUE.<br><br>Has separate transaction control because it needed to have the table updates committed in order to use them in the changelog. |
| PLM | DEV | CALL_MDM_CHANGELOG_DELETE_QUERY | This job identifies all tables with changelog_flag = TRUE. and is scheduled to run every other week. |
| PLM | DEV | MDM_CHANGELOG_DELETE_QUERY | This job calls the stored procedure to process the table that was passed in. It is called from CALL_MDM_CHANGELOG_DELETE_QUERY |

### Future Enhancements

- Still need to enrich the changelog with SVOP data and create the final view
  - Waiting on SVOP to be completed first to prevent rework